3GPP TR 38.901 信道模型软件产品开发计划

第 1 章：引言与架构愿景

1.1 项目任务

本计划旨在为一款基于 Python 的软件库 ChannelModel38901 的开发提供架构设计与详细规格说明。该软件库的核心功能是依据 3GPP 技术报告 TR 38.901，对 5G 及未来无线通信系统的信道进行精确仿真[1]。本开发计划的最终交付成果将具备足够详尽的细节，可作为直接输入，引导大型语言模型（LLM）代码生成工具（如 Gemini-Cli）进行自动化代码实现，从而确保生成代码的准确性、高效性与标准符合性。

1.2 架构支柱

为构建一个健壮、可维护且高性能的仿真库，其软件架构将建立在以下四大支柱之上：

1. **标准遵从性与可验证性 (Standard Adherence and Verifiability)**：软件的任何实现都必须是 3GPP TR 38.901 技术报告中定义的流程和参数的忠实再现。每一个计算步骤、每一个参数取值，都必须能够精确追溯至标准文档中的特定章节、公式或表格[3]。这一点对于确保该库在标准化研发和学术研究中的权威性和可靠性至关重要，正如在 QuaDRiGa 和相关学术工作中强调的那样[4]。
2. **模块化与关注点分离 (Modularity and Separation of Concerns)**：架构将强制实施严格的关注点分离，将系统解耦为逻辑上独立的组件。这种设计借鉴了 ns-3 等成熟仿真平台的成功经验，它将信道建模分解为 ChannelConditionModel（信道状态逻辑）、PropagationLossModel（传播模型）和 ChannelModel（信道生成引擎）等独立模块[6]。具体分离如下：
    - **配置与数据**：管理所有输入参数，如场景定义、天线配置和移动性模型。
    - **信道状态逻辑**：负责判断链路的视距（LOS）/非视距（NLOS）状态。
    - **传播模型**：封装路径损耗和大规模衰落参数的计算逻辑。
    - **信道生成引擎**：负责执行标准中定义的完整、分步骤的信道生成流程。

3. **通过多态实现的可扩展性（策略模式）(Extensibility through Polymorphism - The Strategy Pattern)**：设计将广泛采用抽象基类（Interfaces）和具体的子类实现。例如，为不同的传播场景（如 UMa、UMi、RMa）定义一个统一的抽象接口，然后为每个场景创建具体的实现类。这种设计模式使得未来增加新的场景（如室内 InH、工厂 InF，或 3GPP Release 19 中新增的郊区宏基站 SMa 场景 [5]）变得异常简单，仅需添加新的子类，而无需修改核心的仿真引擎代码。
4. **性能与可伸缩性 (Performance and Scalability)**：考虑到大规模系统级仿真对计算性能的严苛要求，本库的设计将优先采用基于 NumPy 库的向量化操作。通过将循环操作替换为矩阵和向量运算，可以显著提升执行效率。这是高性能科学计算库（如 NVIDIA Sionna 和 MATLAB 5G Toolbox）的通用实践 [9]。

1.3 核心设计哲学：为演进的标准而设计

3GPP 标准并非一成不变的静态文档，而是随着技术演进不断发布新的版本（Release） [11]。例如，TR 38.901 从 Release 14 [13]、Release 16 [3] 到 Release 17 [14]，其参数、模型甚至适用场景都在持续更新和验证，以覆盖新的频段（如 7-24 GHz）和应用 [2]。一个将参数或场景逻辑硬编码在代码中的实现方案，将很快随着标准的更新而过时，导致大量的重构工作，既低效又容易引入错误。

因此，本软件架构的核心设计哲学是**将核心算法逻辑与具体的标准参数和规则解耦**。这一理念直接催生了两个关键的架构决策：参数的外部化存储（详见第 4 章）和基于策略模式的场景化实现（详见第 3 章）。该软件库的长期价值不仅在于实现某一特定版本的标准，更在于其能够以最小的维护成本，快速适应和兼容未来版本的标准。这种设计确保了软件的生命力和长期的可用性。

第 2 章：高层系统架构与公共 API 定义

2.1 简化用户交互：外观（Facade）模式

为提供简洁、易用的编程接口，本库的公共 API 将采用外观（Facade）设计模式。通过提供一个高级接口类 ChannelSimulator，将内部复杂的子系统（包括参数加载器、场景模型、信道生成器等）封装起来。用户无需理解和手动装配底层的十几个对象，即可轻松完成一次完整的信道仿真。这种设计极大地降低了使用门槛，对提升库的采纳率至关重要。

用户的典型工作流程是：1）配置仿真参数；2）运行仿真；3）获取结果。API 的设计将直观地反映这一流程。

2.2 核心 API 规格说明

ChannelSimulator 类是用户与本库交互的主要入口。其公共 API 定义了库与任何客户端应用程序之间的“契约”。

表 2.1: ChannelSimulator 公共 API

方法	参数	返回类型	描述
__init__(self, config_filepath: str)	config_filepath: 指向包含所有仿真参数的 JSON 或 YAML 文件的路径。	None	初始化仿真器实例，从指定的配置文件中加载所有必要的参数和模型。
run_simulation(self)	None	None	为配置中定义的所有基站-用户（BS-UE）链路，执行完整的 12 步信道生成流程。
get_channel_coefficients(self, bs_id: int, ue_id: int)	bs_id, ue_id: 特定链路的基站和用户标识符。	ChannelRealization	返回指定链路的已计算信道实现结果。
get_all_channel_coefficients(self)	None	Dict, ChannelRealization]	以字典形式返回所有已计算的信道实现结果，键为(bs_id, ue_id)元组。

2.3 输入/输出数据契约

为确保数据结构的清晰和一致性，定义了明确的输入和输出数据对象。

- **ConfigurationObject**: 这是一个用于承载所有仿真输入配置的 Python 数据结构（例如，使用 dataclasses 或 pydantic 实现）。它从配置文件中加载，包含了场景选择（如 'UMa'）、网络布局（BS/UE 坐标）、天线阵列配置和移动性参数等。该设计参考了 MATLAB 的 nrPathLossConfig 对象和 5G-Toolkit 的 SimulationLayout 类 [9]。
- **ChannelRealization**: 这是一个用于封装单条链路仿真输出的 dataclass。它将包含最终的信道系数矩阵 H、路径损耗值、LOS/NLOS 状态以及相关的大尺度衰落参数。这种设计为用户提供了干净、自包含且易于解析的仿真结果。

第 3 章：面向对象模型：核心类库与结构

3.1 类图概览

本库的面向对象模型围绕着配置、模型接口、具体实现和生成器这几个核心概念构建。其关系可概念化地描述为：ChannelSimulator（外观）使用一个 ChannelGenerator（引擎），该引擎根据 ScenarioConfig（配置）选择并组合一组策略（如 UMa_Pathloss、UMa_Fading 等具体模型），这些模型在执行时会从 ParameterTables（数据）中读取所需的参数。这种结构通过多态性优雅地处理了不同场景下的逻辑差异。

3.2 配置与参数类

这些类负责持有从用户配置和标准文档中加载的数据。

- class ScenarioConfig: 持有用户定义的静态仿真配置。
    - **属性**: scenario_name: str (例如, 'UMa'), center_frequency_Hz: float, bandwidth_Hz: float, ue_positions: np.ndarray, bs_positions: np.ndarray, ue_velocities: np.ndarray。
- class AntennaArray: 对天线阵列进行建模，其详细参数对于 MIMO 信道至关重要 [3]。
    - **属性**: num_tx_antennas: int, num_rx_antennas: int, tx_geometry, rx_geometry, tx_element_pattern, rx_element_pattern (参考 MATLAB 的 NRAntennaElement [17]), polarization_model: int。
- class ParameterTables: 一个数据容器，专门用于存储从外部文件加载的 3GPP 标准参数表，如 TR 38.901 中的表格 7.4.1-1、7.4.2-1 和 7.5-6 [3]。该对象由 ParameterLoader（见第 4 章）填充。

3.3 核心传播模型接口（抽象基类）

这些抽象基类定义了核心算法的“契约”，是实现策略模式的基础。

- class ABC_ChannelConditionModel(ABC): 定义判断传播条件的接口。
    - @abstractmethod get_condition(self, distance_2d: float, h_ut: float) -> ChannelCondition: 根据二维距离和用户终端高度返回信道状态。
- class ABC_PathlossModel(ABC): 定义路径损耗计算的接口。
    - @abstractmethod calculate_pathloss(self, distance_3d: float, condition: ChannelCondition) -> float: 根据三维距离和信道状态计算路径损耗（dB）。
- class ABC_FadingModel(ABC): 定义大尺度参数（LSP）生成的接口。
    - @abstractmethod generate_lsps(self, condition: ChannelCondition) -> LargeScaleParameters: 根据信道状态生成一组相关的大尺度衰落参数。

3.4 场景化具体实现（具体子类）

这里是策略模式的具体体现，为每个场景提供专门的实现。

- class UMa_ChannelCondition(ABC_ChannelConditionModel): 实现 get_condition 方法，使用 3GPP TR 38.901 表 7.4.2-1 中定义的 UMa 场景 LOS 概率公式 [3]。
- class UMa_Pathloss(ABC_PathlossModel): 实现 calculate_pathloss 方法，使用表 7.4.1-1 中定义的 UMa 场景路径损耗公式 [3]。
- class UMa_Fading(ABC_FadingModel): 实现 generate_lsps 方法，使用表 7.5-6 中定义的 UMa 场景参数，并实现关键的 LSP 互相关逻辑 [3]。
- *将为 UMi 和 RMa 等其他场景定义类似的系列类 (UMi_..., RMa_...)*。

3.5 仿真引擎

- class ChannelGenerator: 核心的流程编排器。它持有当前配置场景下的具体模型实例（例如，一个 UMa_Pathloss 的实例）。其主要方法 generate_channel_realization 将严格按照 3GPP TR 38.901 第 7.5 节中定义的 12 步流程执行 [3]。

3.6 数据表示类

这些类用于在系统内部传递结构化数据。

- @dataclass ChannelCondition: 封装单条链路的传播状态。
  - **属性**: is_los: bool, is_o2i: bool。
- @dataclass LargeScaleParameters: 封装生成的大尺度参数。该数据结构的设计直接来源于 LSP 生成步骤的输出以及 ns-3 中 ParamsTable 所列举的参数 [18]。
  - **属性**: delay_spread_s: float, departure_azimuth_spread_deg: float, arrival_azimuth_spread_deg: float, departure_zenith_spread_deg: float, arrival_zenith_spread_deg: float, k_factor_linear: float, shadow_fade_dB: float。

表 3.1: 核心类职责划分

此表清晰地定义了每个类的单一职责，这是优秀面向对象设计的基石。它为 LLM 提供了一个高层架构“地图”，防止其将多个职责混入单个臃肿的类中。

类名	职责	主要协作者
ChannelSimulator	外观模式实现；提供主要的公共 API。	ScenarioConfig, ChannelGenerator
ScenarioConfig	持有所有用户定义的仿真设置。	ChannelSimulator
ParameterLoader	从外部文件加载 3GPP 标准表格。	ParameterTables

ParameterTables	作为标准定义参数表的内存数据容器。	UMa_Pathloss, UMa_Fading 等
ABC_PathlossModel	路径损耗计算的抽象接口。	-
UMa_Pathloss	UMa 场景路径损耗的具体实现。	ParameterTables
ABC_FadingModel	大尺度参数生成的抽象接口。	-
UMa_Fading	UMa 场景大尺度参数生成的具体实现。	ParameterTables
ChannelGenerator	执行 12 步信道生成流程的核心引擎。	ABC_PathlossModel, ABC_FadingModel

第 4 章：数据模型与标准参数化

4.1 外部化参数原则

本节强制规定，所有源自 3GPP TR 38.901 的数值、公式系数和参数表，**严禁硬编码**在源代码中。所有这些数据必须存储在人类可读、机器可解析的外部文件格式中（推荐使用 JSON）。这一原则是确保软件库长期可维护性和准确性的最关键决策。标准文档中包含数十个表格和数百个参数 [3]，这些参数会随着 3GPP 新版本的发布而修订 [11]。在源代码中查找和修改这些值极易出错。将代码（算法逻辑）与数据（标准参数）分离，是一种基础且稳健的软件工程实践。通过在运行时从外部文件加载参数，用户未来更新到新的 3GPP 标准版本时，可能只需替换一个新的参数文件，而无需修改或重新编译代码，极大地简化了维护和升级过程。

4.2 ParameterLoader 类规格说明

- class ParameterLoader: 此类专门负责读取 JSON 格式的参数文件，并用其内容填充 ParameterTables 数据容器对象。它将包含校验逻辑，以确保加载的配置文件包含了所选场景所需的所有表格和字段，从而保证程序的健壮性。

4.3 JSON 数据结构定义

提供一个清晰的 JSON 文件结构范例，其层级结构将直接映射标准文档中的表格。这为 LLM 提

供了一个具体的数据格式目标。

JSON

```
{
  "version": "16.1.0",
  "scenarios": {
    "UMa": {
      "pathloss_los": {
        "formula": "28.0 + 22 * log10(d3d) + 20 * log10(fc_ghz)",
        "breakpoint_formula": "4 * h_bs_eff * h_ut_eff * fc_hz / c",
        "shadow_fading_std_db": 4.0
      },
      "pathloss_nlos": {... },
      "lsp_parameters_los": {
        "mu_log10_DS": "-6.955 - 0.0963 * log10(fc_ghz)",
        "sigma_log10_DS": 0.66,
        "correlations": {
          "DS_vs_SF": -0.4,
          "ASA_vs_DS": 0.8,
         ...
        }
      },
     ...
    },
    "UMi-StreetCanyon": {... },
    "RMa": {... }
  }
}
```

注意：公式可以作为字符串存储，并使用 Python 的 eval 或专门的解析库进行动态计算，这进一步增强了灵活性。

4.4 TR 38.901 参数到软件对象的映射

下表提供了从标准文档术语到软件内部 LargeScaleParameters 类属性的明确、无歧义的翻译。这是消除 LLM 在实现过程中猜测和误解的关键规范。

表 4.1: TR 38.901 表 7.5-6 参数到 LargeScaleParameters 类的映射 (以 UMa 为例)

TR 38.901 参数	描述	LargeScaleParameters 属性名	数据类型
DS	均方根延迟扩展 (Delay Spread)	delay_spread_s	float
ASD	离开角方位角扩展 (Azimuth Spread of Departure)	departure_azimuth_spread_deg	float
ASA	到达角方位角扩展 (Azimuth Spread of Arrival)	arrival_azimuth_spread_deg	float
ZSD	离开角天顶角扩展 (Zenith Spread of Departure)	departure_zenith_spread_deg	float
ZSA	到达角天顶角扩展 (Zenith Spread of Arrival)	arrival_zenith_spread_deg	float
SF	阴影衰落 (Shadow Fading)	shadow_fade_dB	float
K	莱斯 K 因子 (Ricean K-factor)	k_factor_linear	float

第 5 章：详细实现策略：12 步生成流程

本章是整个计划中规定性最强的部分，它详细描述了 ChannelGenerator.generate_channel_realization 方法内部的实现逻辑。每一步都严格对应 3GPP TR 38.901 第 7.5 节中的流程步骤 [3]。

- **第 1 步: 设置环境、网络布局和天线参数**
  - **逻辑位置**: ChannelSimulator.__init__()
  - **实现**: 此步骤在仿真器初始化时完成。ScenarioConfig 和 AntennaArray 对象根据用户提供的配置文件被实例化并填充。
- **第 2 步: 分配传播条件 (LOS/NLOS)**
  - **逻辑位置**: ChannelGenerator
  - **实现**: ChannelGenerator 遍历所有 BS-UE 链路。对于每个链路，它调用其持有的 ABC_ChannelConditionModel 实例（例如 UMa_ChannelCondition 对象）的 get_condition()方法。该方法根据表 7.4.2-1 中的概率模型，使用随机数生成器来确定 LOS 或 NLOS 状态 [3]。返回的

ChannelCondition 对象被存储，用于后续步骤。

- **第 3 步: 计算路径损耗**
  - **逻辑位置**: ChannelGenerator
  - **实现**: ChannelGenerator 调用其持有的 ABC_PathlossModel 实例的 calculate_pathloss()方法，并将第 2 步得到的 ChannelCondition 作为输入。该方法根据表 7.4.1-1 中的适用公式计算路径损耗 [3]。结果（单位 dB）被存储。
- **第 4 步: 生成大尺度参数 (LSP)**
  - **逻辑位置**: ABC_FadingModel 的具体子类 (例如 UMa_Fading)
  - **实现**: 这是数学上最复杂且最关键的步骤之一，因为它必须正确处理参数间的物理相关性。
    1. **获取参数**: 从 ParameterTables 对象中检索当前场景和条件下的 LSP 均值、标准差以及互相关矩阵 C。
    2. 处理相关性: 直接为每个 LSP 独立生成随机数是错误的，因为它忽略了参数间的物理关联（例如，大的延迟扩展通常与大的角度扩展相关）。正确的做法是：
       a. 生成一个由 N 个独立的标准正态分布（均值为 0，方差为 1）随机变量组成的向量 z。
       b. 对从参数表中获取的 N×N 互相关矩阵 C 进行乔列斯基分解（Cholesky decomposition），得到下三角矩阵 L，使得 C=LLT。
       c. 通过矩阵乘法 y=Lz，将不相关的向量 z 转换为具有正确相关性的随机向量 y。
    3. **缩放与平移**: 对向量 y 中的每个元素 yi，根据对应 LSP 的均值$\mu_i 和标准差\sigma_i$进行缩放和平移，生成最终的 LSP 值。对于对数正态分布的参数（如 DS, ASA 等），计算方式为：LSPvalue=10(yi·σlog10(LSP)+μlog10(LSP))。
    4. **返回结果**: 将所有生成的 LSP 值填充到 LargeScaleParameters 对象中并返回。
- **第 5 步至第 11 步: 簇和径的生成**
  - 这些步骤在 ChannelGenerator 内部按顺序执行，是一系列基于第 4 步生成的 LSP 进行随机变量生成的过程。
  - **第 5 步 (簇延迟)**: 根据指数分布生成簇的延迟，其分布由第 4 步生成的 delay_spread_s 和参数 r_tau 共同决定。
  - **第 6 步 (簇功率)**: 根据簇延迟计算簇的功率，遵循单斜率指数功率延迟分布，并确保总功率归一化。对于 LOS 情况，根据 k_factor_linear 将一部分功率集中分配给第一个到达的簇（视距分量）。
  - **第 7 步 (到达角/离开角)**: 根据拉普拉斯分布（天顶角）和包裹高斯分布（方位角）生成每个簇的角度，分布的扩展度由第 4 步生成的相应角度扩展 LSP（ASA, ASD, ZSA, ZSD）决定。
  - **第 8 步至第 10 步 (簇内耦合, XPR, 初始相位)**: 随机生成簇内多径的属性，包括角度

的耦合、交叉极化功率比（XPR）以及每个径的初始随机相位。
    - 第 11 步 (生成信道系数): 这是最终的信道矩阵合成步骤。对于每个天线对$(u, s)和每个簇 n$，通过对该簇内所有 M 条径的贡献求和来计算信道矩阵 H 的元素。此计算是 3GPP TR 38.901 中公式 7.5-25 的直接实现，需要综合考虑天线方向图、多普勒频移、路径相位和极化等因素。

      Hu,s(t)=n=1ΣNHu,s,n(t)
- **第 12 步: 应用路径损耗和阴影衰落**
    - **逻辑位置**: ChannelGenerator
    - **实现**: 将第 11 步生成的归一化信道矩阵 H 乘以一个缩放因子。该因子由第 3 步计算的路径损耗和第 4 步生成的阴影衰落共同决定，即$\sqrt{P_{rx}/P_{tx}}，其中 P_{rx}$是考虑了路径损耗和阴影衰落后的接收功率。最终得到具有正确绝对功率的信道矩阵。

第 6 章：验证、测试与质量保证

一个未经严格验证的信道模型是不可信的，其仿真结果不具备参考价值。因此，一个全面的测试套件不仅是质量保证的手段，更是模型正确性的活文档。ns-3 等开源项目的开发经验表明，完善的测试是项目成功的关键 [20]。

6.1 单元测试

- **目标**: 验证每个独立模块的功能是否符合标准定义。
- **实施**: 为每个具体的模型类（如 UMa_Pathloss, UMa_ChannelCondition）创建专门的测试文件。测试用例将使用固定的输入值（如距离、频率），并断言其输出与根据 TR 38.901 标准公式 [3] 手动计算的期望值在允许的误差范围内一致。

6.2 集成测试

- **目标**: 验证整个信道生成流程的统计特性是否正确。
- **实施**:
    1. 针对一个特定场景（如 UMa NLOS）执行一次完整的仿真流程。
    2. 进行大量的“投放”（drop），例如生成 10,000 个独立的信道实现。
    3. 收集所有投放生成的大尺度参数（如 DS, ASA, SF 等）。
    4. 计算这些 LSP 样本的统计数据（例如，$\log_{10}(DS)$的均值和标准差）。
    5. 将计算得到的统计数据与 TR 38.901 表 7.5-6 中给出的目标统计值进行比较，断言两

者之间的差异在统计容忍度之内。此测试能够有效地验证第 4 步中复杂的互相关逻辑是否正确实现。

6.3 测试向量

- **目标**: 创建基准快照以防止代码回归。
- **实施**:
  1. 定义一组具有代表性的“典范测试用例”（Canonical Test Cases），例如 UMa LOS @ 500m, UMi NLOS @ 50m。
  2. 为这些用例设置一个固定的随机种子，运行一次仿真，并将完整的输出 ChannelRealization 对象（包含信道矩阵和所有 LSP）序列化并保存为基准“测试向量”。
  3. 在持续集成（CI/CD）流程中，每次代码提交后都重新运行这些测试用例（使用相同的随机种子），并将其输出与已保存的测试向量进行逐位比较。任何不一致都将导致测试失败，从而立即捕获非预期的行为变更。这种验证方法与 3GPP 自身对信道模型的验证思路是一致的 [5]。

第 7 章：针对 LLM 实现的指导方针 (Gemini-Cli)

7.1 推荐技术栈

- **编程语言**: Python 3.10+
- **核心依赖库**:
  - numpy: 用于所有数值计算、数组操作和线性代数（特别是乔列斯基分解）。
  - scipy: 用于特殊数学函数和更复杂的统计分布。
  - pydantic 或 dataclasses: 用于创建类型安全的数据契约类（如 ScenarioConfig, ChannelRealization）。

7.2 编码风格与规范

- **PEP 8**: 必须严格遵守，以保证代码的可读性和一致性。
- **类型提示 (Type Hinting)**: 所有函数签名、类属性和变量都必须包含明确的类型提示。这对于 LLM 理解代码意图和数据流至关重要。
- **文档字符串 (Docstrings)**: 每个类和公共方法都必须有 Google 风格的文档字符串，清晰地

解释其功能、参数、返回值和使用示例。

7.3 分阶段实施指令

为引导 LLM 逐步构建整个系统，建议采用以下分阶段的开发指令：

- **阶段一：数据模型与参数加载**
  - **指令**: “请根据第 3 章和第 4 章的定义，实现 ScenarioConfig, AntennaArray, ParameterTables, ChannelCondition, LargeScaleParameters 这些数据类。然后，实现 ParameterLoader 类，使其能够读取第 4.3 节中定义的 JSON 文件结构，并将数据填充到 ParameterTables 和 ScenarioConfig 对象中。为 ParameterLoader 编写单元测试，验证其能正确加载和校验数据。”
- **阶段二：传播模型实现**
  - **指令**: “请根据第 3.3 节实现 ABC_ChannelConditionModel, ABC_PathlossModel, ABC_FadingModel 这些抽象基类。然后，根据第 3.4 节，为 UMa 场景实现具体的子类 UMa_ChannelCondition, UMa_Pathloss 和 UMa_Fading。为 UMa_Pathloss 和 UMa_ChannelCondition 编写单元测试，使用标准文档中的公式验证其计算结果的准确性。”
- **阶段三：仿真引擎核心逻辑**
  - **指令**: “请实现 ChannelGenerator 类。在该类中，实现 generate_channel_realization 方法，严格遵循第 5 章详述的 12 步信道生成流程。重点关注第 4 步中基于乔列斯基分解的 LSP 生成逻辑。同时，编写第 6.2 节中描述的集成测试，以验证生成 LSP 的统计特性符合标准。”
- **阶段四：API 封装与优化**
  - **指令**: “请根据第 2.2 节的 API 规格说明，实现 ChannelSimulator 外观类。将所有底层对象的创建和调用封装在该类内部。最后，审查并优化 ChannelGenerator 中所有基于 NumPy 的计算，确保其性能高效。完成所有文档字符串和类型提示。”

Works cited

1. 3GPP TR 38.901 Channel Model - ResearchGate, accessed August 2, 2025, https://www.researchgate.net/profile/Qiuming-Zhu-2/publication/348790466_3GPP_TR_38901_Channel_Model/links/633781d376e39959d6898826/3GPP-TR-38901-Channel-Model.pdf
2. Next G Alliance looks at channel models in 3GPP TR 38.901, accessed August 2, 2025, https://www.3gpp.org/news-events/partner-news/channel-measurements
3. TR 138 901 - V16.1.0 - 5G; Study on channel model for frequencies from 0.5 to 100 GHz (3GPP TR 38.901 version 16.1.0 Release 16 - ETSI, accessed August 2, 2025, https://www.etsi.org/deliver/etsi_tr/138900_138999/138901/16.01.00_60/tr_138901v160100p.pdf

4. stjaeckel/QuaDRiGa-NG: QuaDRiGa, short for QUAsi Deterministic Radlo channel GenerAtor, is used for generating realistic radio channel impulse responses for system-level simulations of mobile radio networks. - GitHub, accessed August 2, 2025, https://github.com/stjaeckel/QuaDRiGa-NG
5. Overview of 3GPP Release 19 Study on Channel Modeling Enhancements to TR 38.901 for 6G - arXiv, accessed August 2, 2025, https://arxiv.org/html/2507.19266v1
6. ns3::ThreeGppChannelConditionModel Class Reference - ns-3, accessed August 2, 2025, https://www.nsnam.org/docs/release/3.34/doxygen/classns3_1_1_three_gpp_channel_condition_model.html
7. (PDF) ns-3 Implementation of the 3GPP MIMO Channel Model for Frequency Spectrum above 6 GHz - ResearchGate, accessed August 2, 2025, https://www.researchgate.net/publication/313796828_ns-3_Implementation_of_the_3GPP_MIMO_Channel_Model_for_Frequency_Spectrum_above_6_GHz
8. ns3::ThreeGppChannelModel Class Reference - ns-3, accessed August 2, 2025, https://www.nsnam.org/docs/release/3.35/doxygen/classns3_1_1_three_gpp_channel_model.html
9. nrPathLossConfig - Path loss configuration parameters - MATLAB - MathWorks, accessed August 2, 2025, https://www.mathworks.com/help/5g/ref/nrpathlossconfig.html
10. Python For Wireless Digital Communications - Researcherstore, accessed August 2, 2025, https://researcherstore.com/courses/python-for-wireless-digital-communications/
11. Specification # 38.901 - 3GPP, accessed August 2, 2025, https://www.3gpp.org/dynareport/38901.htm
12. 3GPP specification series: 38series, accessed August 2, 2025, https://www.3gpp.org/dynareport/38-series.htm
13. TR 138 901 - V14.0.0 - 5G; Study on channel model for frequencies from 0.5 to 100 GHz (3GPP TR 38.901 version 14.0.0 Release 14) - ETSI, accessed August 2, 2025, https://www.etsi.org/deliver/etsi_tr/138900_138999/138901/14.00.00_60/tr_138901v140000p.pdf
14. ETSI TR 138 901 V17.0.0 (2022-04) - iTeh Standards, accessed August 2, 2025, https://cdn.standards.iteh.ai/samples/65119/ebabf061f4f84558b2f093a710068bdf/ETSI-TR-138-901-V17-0-0-2022-04-.pdf
15. [2504.15589] Validation of 3GPP TR 38.901 Indoor Hotspot Path Loss Model Based on Measurements Conducted at 6.75, 16.95, 28, and 73 GHz for 6G and Beyond - arXiv, accessed August 2, 2025, https://arxiv.org/abs/2504.15589
16. Channel Models — 5G Toolkit R24a documentation, accessed August 2, 2025, https://gigayasawireless.github.io/toolkit5G/api/5G_Toolkit/ChannelModels/channelModels.html

17. phased.NRAntennaElement - 5G antenna element described in 3GPP TR 38.901 specification - MATLAB - MathWorks, accessed August 2, 2025, https://www.mathworks.com/help/phased/ref/phased.nrantennaelement-system-object.html
18. ns-3: ns3::ThreeGppChannelModel::ParamsTable Struct Reference - QKDNetSim (v2.0), accessed August 2, 2025, https://www.qkdnetsim.info/html/d4/d26/structns3_1_1_three_gpp_channel_model_1_1_params_table.html
19. ns3::ThreeGppChannelModel::ParamsTable Struct Reference - ns-3, accessed August 2, 2025, https://www.nsnam.org/doxygen/d4/d26/structns3_1_1_three_gpp_channel_model_1_1_params_table.html
20. GSOC2019ThreeGPPChannel - Nsnam - ns-3, accessed August 2, 2025, https://www.nsnam.org/wiki/GSOC2019ThreeGPPChannel
21. Test Zone Validation for FR2 MPAC Using Directional Antenna Based Virtual Array - Aalborg Universitets forskningsportal, accessed August 2, 2025, https://vbn.aau.dk/files/697659098/Accepted_version.pdf